SOFTWARE AND REPRODUCIBILITY FOR NUMERICAL EXPERIMENTS ON PROJECTED MULTILINEAR MODELS

ELIUTH CHAVERO JASSO

Abstract. We describe the software supporting two independent computational studies of multilinear maps under orthogonal projection, and the measures taken to make their reported results reconstructible. The first study evaluates finite composition trees of multilinear maps whose intermediate outputs are recursively projected, and computes upper bounds and rigorous interval enclosures for the resulting error [1]. The second fits cyclically symmetric multilinear maps together with orthogonal projectors and measures fourteen diagnostic quantities [2]. The two share no code and no evidence; they are presented here as two case studies of one set of reproducibility practices, not as one system.

We describe: how multilinear maps, typed trees and projectors are represented; why each study carries a reference implementation alongside an optimised one, and what is compared between them; how numerical precision is controlled, including a process-global reduced-precision setting that silently defeats such control and how it is contained; how a mathematical configuration is distinguished from an execution of it, so that repeated and resumed executions are never counted as independent observations; what each run records; and how the tables, figures and manuscripts are regenerated. We report what has been verified — 73 and 85 automated tests, agreement between reference and optimised implementations, agreement between two devices in double precision — and what has not: in particular, no reproduction of either study from a clean environment by an independent party has been performed.

[2020]68N30, 65Y05, 65G50, 15A69 reproducible research, numerical software, provenance, interval arithmetic, CPU/GPU agreement, experiment identity

1. Scientific purpose of the software

Two questions are being asked, by two disjoint bodies of code.

Case study I: bounds for recursively projected composition trees. Given a finite tree whose internal vertices carry multilinear maps between finite-dimensional Hilbert spaces, and orthogonal projectors onto prescribed subspaces at every vertex, how large can the discrepancy between unprojected and recursively projected evaluation be? The software enumerates trees exactly, evaluates both quantities, computes four families of upper bound, and produces rigorous interval enclosures of explicit lower constructions. Its scientific role is to *check* the estimates of [1] on many configurations, and to *search* for constructions that approach them. It cannot prove anything, and no bound it computes is a proof.

Case study II: diagnostics for fitted multilinear maps and projectors. Given a cyclically symmetric multilinear map in a low-rank representation together with a fitted orthogonal projector, what do fourteen diagnostic quantities — projector identities, a commutator residual, closure and associator defects, principal angles between subspaces at different resolutions, mode ranks, and a generalised Jacobi residual — take as values? Its scientific role is to *measure*, and in four cases the measurement is a negative result [2].

Remark 1.1 (The two case studies are kept apart). No module is shared, no dataset is shared, and no result from one is used as evidence for the other. They appear in one article because they are

Date: 31 July 2026.

This article makes no independent mathematical claim. Every mathematical statement it refers to belongs to [1] or [2], and is cited there.

governed by one set of reproducibility practices, and because a reader wishing to rebuild either will follow the same steps. Any future connection between them would require a theorem, and none exists.

2. Mathematical objects and their representation

2.1. **Case study I.**

object	representation
typed space	ambient dimension, reduced dimension, isometry Q, projector $P = QQ^*$; the identities $P^2 = P$, $P^* = P$ and $Q^*Q = I$ are validated on construction
multilinear map	ordered input signature, output type, and a dense coefficient tensor whose shape must agree with the signature
leaf	a label and a type; the label must resolve to reduced input data of the declared size
internal vertex	a map identifier, an output type, and ordered children whose output types must equal the corresponding input types
tree	an immutable structure with a canonical serialisation and a content hash

Validation precedes evaluation. An edge whose types do not match is rejected before any arithmetic occurs; it is a control, not a numerical failure. Forty-eight such configurations are declared and all are rejected.

2.2. **Case study II.** A multilinear map is stored in a rank-R canonical polyadic representation [5], which avoids materialising a coefficient tensor of size n^{a+1}. The projector is $P = UU^*$ for an isometry $U \in \mathbb{C}^{n\times r}$. Cyclic symmetrisation is applied by an orthogonal projector onto the cyclically invariant subspace. A reduced tensor is obtained by contracting the map against U in every slot.

Remark 2.1 (Representation is not structure). A low-rank representation is a storage device. It is not unique — factors may be rescaled subject to a product constraint and permuted — so any comparison of factors must be performed modulo that freedom. This is why the comparison tooling of [2] reports several aligned quantities separately rather than one score.

3. Reference and optimised implementations

Both case studies carry two implementations of their core computation, and compare them. The purpose is to detect errors that a single implementation cannot reveal.

Case study I.. A reference evaluator applies each coefficient tensor with explicit index loops, in the most direct transcription of the definition. An optimised evaluator contracts modes through vectorised operations [4], and a third runs on a GPU [6]. All three are exercised on real and complex data, on arities two through four, with repeated leaves, and against exactly invariant controls for which the correct answer is known in closed form.

Case study II.. A reference implementation of the reduced tensor uses explicit loops; an optimised one uses a contraction exploiting the low-rank structure, including the cyclic average. They agree to better than 10^{-10} in double precision and 10^{-4} in single precision, and an exact rational case at $n = 2$, $r = 1$ is checked against a hand computation with no floating-point arithmetic involved.

Fresh implementation rather than reuse. The core numerical primitives of case study II were reimplemented rather than imported from the historical code, for the reason given in Section 4. The reimplementation is checked against the historical formulas by direct citation of the source lines it reproduces.

4. Numerical precision and hardware controls

4.1. **A process-global setting that defeats precision control.** The historical code sets, unconditionally and at import time when a GPU is present, two process-global flags: one enabling

reduced-precision matrix multiplication on the GPU (TF32), and one lowering the default precision of single-precision matrix multiplication. Neither is scoped to the importing module.

The consequence is that a process which imports that module for any reason has its own precision discipline silently overridden, whatever it set beforehand. This is not a hypothetical: it is why the primitives of case study II were reimplemented rather than imported. The reimplementation exposes a single idempotent entry point that establishes the required precision settings, together with an assertion that re-checks them, so that the discipline can be re-established at any point rather than assumed.

4.2. **A second hazard: moving a model between devices.** Transferring a model to another device with the framework's standard method updates the tensors but not plain attributes recording the intended device and data type. Code reading those attributes then disagrees with the tensors. Device transfers are therefore performed by constructing an instance on the target device and copying parameter values explicitly, not by the standard method.

4.3. **What is compared, and how.** Both case studies compare a CPU and a GPU execution of the same configuration, in double precision, with reduced-precision matrix multiplication explicitly disabled and deterministic algorithms enabled.

quantity	CPU	GPU	note
idempotence residual	1.312×10^{-15}	1.198×10^{-15}	both at the noise floor
self-adjointness residual	0 (exact)	2.220×10^{-16}	both at the noise floor
unexplained relative residual	1.0000380759060201	1.0000380759060197	relative difference 4.4×10^{-16}

Only the third quantity is large enough for a relative comparison to mean anything; the first two are at the noise floor on both devices, so their agreement establishes that neither device is producing something structurally wrong, and no more. For case study I the largest observed difference between the CPU and GPU evaluators over the whole registry was 1.922×10^{-8}, which is a validation threshold on the implementation and not a quantity appearing in any theorem.

Remark 4.1 (Timing is not compared here)*.* A single timing comparison at one problem size is not a hardware-performance result. The throughput question is answered instead by the swept measurement of [2], over 160 configurations and four ambient dimensions, and even that is a measurement on one machine.

5. Experiment specification

An experiment is specified by a declarative configuration: the arity, the ambient dimension, the reduced rank, the representation rank, the seed, the device, the numerical precision, the objective, the number of optimisation steps, and the tolerances. The configuration is resolved and validated before execution, and the resolved form — not the form as written — is what is recorded.

5.1. **Configuration, execution, restart.** This distinction is the one that most affects how results may be reported.

identity	meaning
configuration	one distinct mathematical setting. Two runs with the same arity, dimension, ranks and seed are the same configuration, whatever hardware they used
execution	one run of one configuration. Distinguished by device, by time, and by whether it resumed from a checkpoint
optimiser restart	one search trajectory within an execution

The ledger records both identifiers for every run, together with the parent execution when a run resumed from a checkpoint. This is what makes the following statements checkable rather than assertions: that the 416 executions of the swept extension in [2] cover 208 configurations, each

executed once per device; and that the nineteen historical runs of the same study form one chain of resumptions rather than nineteen independent trials.

Remark 5.1 (Why this matters for reporting). Counting executions as configurations inflates an apparent sample size by whatever factor the execution grid happens to have. Counting optimiser restarts as replicates does the same and additionally suggests an uncertainty estimate that does not exist. Both are prevented here by the identity scheme rather than by discipline in writing.

5.2. **Work that was specified but not executed.** Case study I specifies an extended optimiser grid of 460,800 search trajectories that was not executed, under a stated computational-cost constraint. Case study II specifies two further sweep stages that were not executed. In both, the unexecuted work is recorded in the configuration and marked as not executed. It is not omitted, and the executed portion is never presented as though it were the whole.

6. Provenance

6.1. **What a run records.** Every run directory contains: the resolved configuration; the exact command; the software environment and dependency versions; the hardware; the source commit; content hashes of the mathematical objects and of the input data; the input tensors; the computed metrics; any enclosures; the logs; and hashes of every output file. History is append-only: a retry increments a counter and adds a record, and never rewrites an earlier one.

6.2. **Historical evidence is ingested, not trusted.** The historical evidence for case study II was ingested by a tool that hashes each source file, hashes each run directory, parses the run log, cross-references it against the on-disk summaries, reconstructs the chain of resumptions, and reclassifies every historical run against the current vocabulary. That process is what established the provenance findings reported in [2]: that every historical run was exploratory despite directory names suggesting otherwise, that nine logged runs form a single chain, and that the analysis script changed mid-campaign without this being recorded.

6.3. **A corrected classification, re-derived rather than re-run.** The classification of enclosure outcomes in case study I was found to be defective: the label reserved for an exactly determined constant was assigned only when the proved upper bound was itself zero — the vacuous case — while configurations whose lower and upper bounds genuinely coincided at a positive value received a weaker label, and configurations for which no positive lower bound had been obtained were described as having a certified lower bound.

The classification is a pure function of the certified lower and upper bounds, both of which are recorded. It was therefore corrected in the source and *re-derived* from the recorded bounds, without re-running any experiment. The re-derivation is a separate script, its output is written alongside the original rather than over it, and the correspondence between the old and new labels is a bijection on the agreement classes, so the two derivations can be compared directly. Historical outputs were not modified.

7. Reconstructing the tables, figures and manuscripts

7.1. **Tables.** Tables in [1] are generated from the recorded registries by a script that also emits the numerical macros used in the text, so that no number in the manuscript is typed by hand. Where the generated text carried implementation vocabulary, a separate and auditable rewriting step replaces it with standard terminology; that step touches headings and labels only, never a numeric value, and it refuses to complete if any forbidden token survives.

7.2. **Figures.** Figures in [3] are produced by a single generator from committed data files. It emits each figure as a vector PDF, an SVG and a 300-dpi PNG, and writes a manifest recording, per figure: the input files with checksums, the output files with checksums, the checksum of the generator itself, and the versions of the numerical libraries. No figure value is entered by hand.

7.3. **Manuscripts.** Each manuscript builds from a clean directory with a standard LaTeX toolchain. The build is checked for undefined references, undefined citations, overfull lines and errors, and the resulting PDF is checked for correct text extraction — specifically, that ligatures are recoverable, which requires an outline font rather than the bitmap fonts that a bare `T1` font encoding selects by default.

8. Testing strategy

suite	tests	scope
case study I	73	typed validation, enumeration, evaluator agreement, bound soundness, ordering rule against all permutations through arity seven, interval enclosures, run-artifact contract
case study II	85	projector identities, block-level diagnostics, gauge invariance, ledger semantics including retry and checkpoint lineage, and the enforcement point below

Both suites pass. Two categories of test deserve mention.

Negative controls that must fail. Invalid typed edges must be rejected; a deliberately sign-mutated identity must be detected; an objective claimed to be invariant under a change of basis must be checked against an actual change of basis. Each of these found a real defect during development, which is the reason they exist.

Enforcement rather than convention. One function is the single point at which a run's evidential status is assigned, and it raises an exception — rather than silently downgrading — if an exploratory run is ever assigned a status reserved for a rigorous one. A schema-drift detector pinned to file checksums was verified by deliberately editing a schema, confirming the test failed, restoring the file, and confirming it passed again.

Remark 8.1 (What passing tests establish). Passing tests establish consistency between the implementation and its specification. They do not establish any mathematical statement in either companion article, and no bound computed by this software is a proof of anything.

8.1. **Expanded coverage and a clean-room run (follow-up session).** A follow-up session expanded case study I to 132 tests (up from 73) and added an independent 18-test suite covering the six-term signed-combination resolution (two structurally unrelated symbolic implementations, four mutation tests), the exact $k = 2$ and $k = 3$ closed-form constructions across four dimension/rank pairs each, the verified Markov construction, and configuration/execution-identity checks on a companion applied-benchmark's raw results (no duplicate (topology, seed, budget, method) identities; every configuration ran the same declared method set). A separate 16-test suite covers the applied benchmark itself (the adaptive rank-allocation companion article): budget conservation for every allocation method and ablation, projector orthonormality and idempotence, no test-set leakage, determinism given a fixed seed, and regression tests for two real bugs found and fixed during that work (one ablation that could not perturb its own greedy ranking; one network topology that made a node's rank irrelevant by construction).

A fresh Docker container, built from an unrelated public base image with no cached state from the author's machine, reran both existing suites plus the new ones, plus the six-term resolution and exact closed-form scripts, plus a full re-execution of the applied benchmark's synthetic validation level – byte-identical, by SHA-256, to the already-committed results. One real environment gap (a missing `git` binary, needed by tests that record commit-hash lineage) was found and fixed before the container run was declared passing. Full log in `clean_room/reproduction_run/` in the accompanying repository.

9. Cross-platform and cross-implementation agreement

Three kinds of agreement are checked and are reported separately:

(1) between a reference implementation and an optimised one, on the same device (Section 3);
(2) between two devices, on the same configuration in double precision (Section 4);
(3) between two independent implementations of the same mathematical quantity written against different formulations (case study II's reduced tensor).

For each, we report the quantity compared, the precision, the tolerance, the absolute difference and, where the magnitudes make it meaningful, the relative difference. Where a quantity is at the numerical noise floor on both sides, we say so and draw no further conclusion.

10. Limitations

(1) **No clean-environment reproduction has been performed.** Neither study has been rebuilt and re-run from a fresh environment as a dedicated exercise, by the author or by anyone else. Continuous integration does exercise a clean install of the packaged source, but it does not then run either study's full experiment set inside that environment. This is the largest outstanding gap.
(2) **No independent party has reproduced anything.** All the agreement reported above is internal.
(3) Numerical results come from one machine and one software stack. Timing results in particular do not transfer.
(4) The extended optimiser grid of case study I and the two further sweep stages of case study II were specified and not executed.
(5) Multilinear operator norms are computed exactly only for declared explicit constructions; elsewhere a Frobenius upper bound is substituted, which inflates the constants that depend on it.
(6) Ten of the fourteen experiments of case study II were not extended over a parameter grid and were not run on a GPU at all.
(7) The numerical evidence of case study I was produced at an earlier source commit than the manuscript text. The provenance file records both, and the corrected classification of Section 6 was re-derived from the recorded bounds rather than regenerated by re-running the pipeline at the later commit.

11. Instructions for independent reproduction

The steps below are what a third party would follow. They have been exercised individually but not as one continuous pass from a fresh environment; see Section 10.

(1) **Environment.** Create an isolated Python environment at the version recorded in the provenance file and install the locked dependency set. A GPU is optional; every result except the device-agreement measurements can be reproduced without one.
(2) **Tests.** Run both test suites and confirm the counts in Section 8. A failure here means the environment differs from the recorded one, and nothing further should be attempted until it is resolved.
(3) **Data.** Either regenerate the experiment records by running the pipelines with the recorded configurations, or use the records shipped with the package. Regeneration is the stronger check and takes substantially longer; the shipped records carry checksums so that the two can be compared.
(4) **Tables and macros.** Run the table generator and the classification re-derivation. Both are deterministic and idempotent: running either twice produces byte-identical output.

(5) **Figures.** Run the figure generator. It writes a manifest with checksums, which should be compared against the shipped manifest.
(6) **Manuscripts.** Build each manuscript from a clean directory and confirm zero undefined references, zero undefined citations, zero overfull lines and zero errors, and confirm that text extraction from the resulting PDF recovers ligatures correctly.
(7) **Checksums.** Verify the package checksum file.

12. Statement on the use of AI tools

An AI assistant was used in preparing the software and the manuscripts: for drafting and editing prose, for restructuring the manuscripts, for locating defects in the classification code and in reported numerical values, and for writing auxiliary scripts. All mathematical statements, all proofs, all citations, all interpretations and the final wording are the author's responsibility, and the author has checked them. No AI system is an author. Any citation suggested by such a tool was verified against the primary source before being retained. Authors submitting elsewhere should consult the target venue's current policy on AI-assisted writing, which this statement is intended to satisfy but may not, depending on the venue.

References

1. Eliuth Chavero Jasso, *Error propagation under recursive orthogonal projection of finite multilinear composition trees*, 2026, Companion article; case study I of this software article.
2. ———, *A numerical study of cyclic multilinear maps, orthogonal projectors, and multiresolution tensor representations*, 2026, Companion article; case study II of this software article.
3. ———, *Supplementary numerical results for projected multilinear models*, 2026, Supplement to the numerical study.
4. Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant, *Array programming with NumPy*, Nature **585** (2020), 357–362.
5. Tamara G. Kolda and Brett W. Bader, *Tensor decompositions and applications*, SIAM Review **51** (2009), no. 3, 455–500.
6. Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala, *PyTorch: An imperative style, high-performance deep learning library*, Advances in Neural Information Processing Systems 32, 2019, pp. 8024–8035.

Independent Researcher, Apizaco, Tlaxcala, Mexico